

# Implementation guide

## Implementation & Considerations:

For this project main area was deploying a honeypot on a cloud environment accessible to hackers. For this reason I had to choose between multiple cloud providers , AWS, And azure were the best options. From them I chose Azure because of the free 200 credits , which were enough for me to complete the project and analysis.

This part is where I will implement the VM's fpor both management and Honeypot.

Anyone who is interested can go through this workflow in deploying this honeypot.

## Advantages of Hosting on AZURE:

Had several benefits but the best ones were:

- Cost was low with 200 \$ starting credits and if you are a student you qualify for 100 \$ more.
- Easily adjustable machine specifications like RAM to match different workloads , Really good for scalability.
- Azures built in Firewall protects sensitive data.
- Azure had the lowest downtime best for applications such as honeypots.
- Simplified GUI streamlines and simplifies the whole deployment process.

To sum up, Azure was the best practical and efficient choice for this honeypot.

## Resource group:

A dedicated resource group is needed to contain all project components( VM's network, NSG , Ips)

### Steps:

- 1- After creation of account just choose Create a resource or search resource groups.
- 2- You chose what subscription model you want , And you can name the resource group here
- 3- Selecting a region , You choose a region with your specific requirements, Remember some resources are only limited to some regions so. You choose region depending on your requirements , For Us Canada Central was the best option.
- 4- Press review and Create.

[Home](#) >

## Create a resource group

× Basics   Tags   Review + create

**Resource group** - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Subscription \* ⓘ

Resource group name \* ⓘ

× A resource group with the same name already exists in the selected subscription Azure subscription 1.

Region \* ⓘ

[Previous](#) [Next](#) [Review + create](#)

### **Virtual Network and Subnet:**

Both VM's (Management and Honeypot) will be inside this dedicated network. In best conditions would be 3 VM's , 3<sup>rd</sup> being the actual financial decoy layer.

#### **Steps:**

- 1- Search Virtual networks in azure.
- 2- Click Create a New
- 3- Add in your resource group, And name it . With selecting the region best is to choose the same region as your resource group.

# Create virtual network ...

Basics   Security   IP addresses   Tags   Review + create

Azure Virtual Network (VNet) is the fundamental building block for your private network in Azure. VNet enables many types of Azure resources, such as Azure Virtual Machines (VM), to securely communicate with each other, the internet, and on-premises networks. VNet is similar to a traditional network that you'd operate in your own data center, but brings with it additional benefits of Azure's infrastructure such as scale, availability, and isolation.

[Learn more.](#)

## Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription \*

Azure subscription 1

Resource group \*

HP-NETWORKSECURITY

[Create new](#)

## Instance details

Virtual network name \*

Region \* ⓘ

(Canada) Canada Central

[Deploy to an Azure Extended Zone](#)

4- In IP address create your subnets. And review and create.

## Create virtual network ...

✖ Basics   Security   IP addresses   Tags   Review + create

Configure your virtual network address space with the IPv4 and IPv6 addresses and subnets you need. [Learn more](#)

Define the address space of your virtual network with one or more IPv4 or IPv6 address ranges. Create subnets to segment the virtual network address space into smaller ranges for use by your applications. When you deploy resources into a subnet, Azure assigns the resource an IP address from the subnet. [Learn more](#)

☐ Allocate using IP address pools. [Learn more](#)

+ Add a subnet

10.0.0.0/16 [Delete address space](#)

This address prefix overlaps with virtual network 'honeynet-vnet'. If you intend to peer these virtual networks, change the address space. [Learn more](#)

10.0.0.0 - 10.0.255.255      65,536 addresses

| Subnets                 | IP address range      | Size                | NAT gateway                                   |
|-------------------------|-----------------------|---------------------|-----------------------------------------------|
| <a href="#">default</a> | 10.0.0.0 - 10.0.0.255 | /24 (256 addresses) | - <a href="#">edit</a> <a href="#">delete</a> |

Add IPv4 address space | ▾

Previous

Next

Review + create



## Network Security Groups:

This gives a controlled environment where all inbound traffic is either:

- Restricted to our own management IPs (for SSH/RDP)
- Intentionally open to the Internet for honeypot services.

You would need these equal to the number of VM's You host, Each NSG being linked to its corresponding VM.

### Steps:

- 1- Search Network Security Groups in azure.
- 2- Click Create a New
- 3- Add in your resource group, And name it . With selecting the region best is to choose the same region as your resource group.

The screenshot shows the 'Create network security group' page in the Azure portal. The breadcrumb navigation at the top reads 'Home > Network foundation | Network security groups >'. The main heading is 'Create network security group' followed by an ellipsis. Below the heading are three tabs: 'Basics' (selected), 'Tags', and 'Review + create'. Under the 'Basics' tab, there are two sections: 'Project details' and 'Instance details'. In the 'Project details' section, 'Subscription \*' is set to 'Azure subscription 1' and 'Resource group \*' is empty, with a 'Create new' link below it. In the 'Instance details' section, 'Name \*' is empty and 'Region \*' is set to 'Canada Central'. At the bottom of the page, there is a blue 'Review + create' button, a '< Previous' button, a 'Next : Tags >' button, and a link 'Download a template for automation'.

### Deploying Management VM:

To avoid exposing administrative interfaces with Tpot and decoys, You make a dedicated Management VM. This acts as a jump box to access T-pot Honeypot via ssh and webUI. We run

AZURE CLI on this for troubleshooting and managing , and keeping our laptop less exposed to direct internet facing resources.

### Steps:

- 1- Search Virtual Machines in azure.
- 2- Click Create a New, Choose Virtual Machine from options.
- 3- Add in your resource group, And name it . With selecting the region best is to choose the same region as your resource group.
- 4-Choose an image, For me I used Ubuntu Server 24.04 X64 GEN2, with size for 2vcpus , and 8gb memory.
- 5- Choose SSH public key, and generate a new key pair, Or you could choose a password and user for simplicity.

Home > Compute infrastructure | Virtual machines >

## Create a virtual machine

Help me choose the right VM size for my workload Help me create a V

Help me create a low cost VM Help me create a VM optimized for high availability Help me choose the right VM

Security type Trusted launch virtual machines  
[Configure security features](#)

Image \* Ubuntu Server 24.04 LTS - x64 Gen2  
[See all images](#) | [Configure VM generation](#)

VM architecture ☐ Arm64 ☒ x64

Run with Azure Spot discount ☐

Size \* Standard\_D2s\_v3 - 2 vcpus, 8 GiB memory (\$81.03/month)  
[See all sizes](#)

Enable Hibernation ☐  
 Hibernate does not currently support Trusted launch and Confidential virtual machines for Linux images. [Learn more](#)

**Administrator account**

Authentication type ☒ SSH public key ☐ Password

Azure now automatically generates an SSH key pair for you and allows you to store it for future use. It is a fast, simple, and secure way to connect to your virtual machine.

Username \* azureuser

SSH public key source Generate new key pair

SSH Key Type ☒ RSA SSH Format ☐ Ed25519 SSH Format  
 Ed25519 provides a fixed security level of no more than 128 bits for 256-bit key, while RSA could offer better security with keys longer than 3072 bits.

< Previous Next : Disks > Review + create

- 6- Go to networking tab and connect to VNET you created.



Home > Compute infrastructure | Virtual machines >

Create a virtual machine

Help me choose the right VM size for my workload

+2

Help me create a low cost VM

Help me create a VM optimized for high availability

Help me choose the right VM size for my workload

Basics

Disks

Networking

Management

Monitoring

Define network connectivity for your virtual machine by configuring network interface card (NIC) settings. You can control ports, inbound and outbound connectivity with security group rules, or place behind an existing load balancing solution. [Learn more](#)

### Network interface

When creating a virtual machine, a network interface will be created for you.

Virtual network \*

Create new

Public IP

None

Create new

NIC network security group

☐ None

☒ Basic

☐ Advanced

Public inbound ports \*

☐ None

☒ Allow selected ports

Select inbound ports \*

SSH (22)

This will allow all IP addresses to access your virtual machine. This is only recommended for testing. Use the Advanced controls in the Networking tab to create rules to limit inbound traffic to known IP addresses.

Estimated monthly cost

**\$81.03** / month

[View cost details](#)

[Give feedback about your estim](#)

< Previous

Next : Management >

Review + create

7- Leave everything else as could be set later and is not really needed, Review and create and create this VM.

## Deploying Honeypot VM:

This is the VM for your honeypot deployment and all ports would be left open here , Use the same process as previous but use Password and username instead as the Honeypot would delete the SSH keys before you open it and will crash when you try logging in through ports.

```
--resource-group HP-NETWORKSECURITY \  
--name tpot-vm \  
--vnet-name hp-vnet \  
--subnet hp-subnet \  
--image UbuntuLTS \  
--size Standard_D4s_v3 \  
--admin-username mojiz \  
--generate-ssh-keys \  
--public-ip-sku Standard \  
--nsg hp-nsg
```

### ***Installing honeypot framework:***

1- Install Git and clone T-Pot repository

*sudo apt install git -y*

*git clone <https://github.com/telekom-security/tpotce.git>*

2- Running installer

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
oppenheimer@abp-Tower:~$ sudo apt install git -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
git is already the newest version (1:2.43.0-1ubuntu7.3).  
git set to manually installed.  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.  
oppenheimer@abp-Tower:~$ sudo git clone https://github.com/dtag-dev-sec/tpotce  
Cloning into 'tpotce'...  
remote: Enumerating objects: 17644, done.  
remote: Counting objects: 100% (123/123), done.  
remote: Compressing objects: 100% (61/61), done.  
remote: Total 17644 (delta 81), reused 62 (delta 62), pack-reused 17521 (from 3)  
Receiving objects: 100% (17644/17644), 327.08 MiB | 60.70 MiB/s, done.  
Resolving deltas: 100% (9828/9828), done.  
oppenheimer@abp-Tower:~$
```

*cd tpotce*

`sudo ./install.sh --type=user`

```
oppenheimer@abp-Tower:~$ cd tpotce/iso/installer
-bash: cd: tpotce/iso/installer: No such file or directory
oppenheimer@abp-Tower:~$ cd tpotce
ls
CHANGELOG.md SECURITY.md docker genuser.sh uninstall.sh
CITATION.cff compose docker-compose.yml genuserwin.ps1 update.sh
LICENSE deploy.sh dps.ps1 install.sh version
README.md doc env.example installer
oppenheimer@abp-Tower:~/tpotce$ cd installer
oppenheimer@abp-Tower:~/tpotce/installer$ sudo ./install.sh --type=user
sudo: ./install.sh: command not found
oppenheimer@abp-Tower:~/tpotce/installer$ ls
install remove
oppenheimer@abp-Tower:~/tpotce/installer$ sudo ./install --type=user
sudo: ./install: command not found
oppenheimer@abp-Tower:~/tpotce/installer$ ls -l
total 8
drwxr-xr-x 2 root root 4096 Nov 29 12:48 install
drwxr-xr-x 2 root root 4096 Nov 29 12:48 remove
oppenheimer@abp-Tower:~/tpotce/installer$ cd install
oppenheimer@abp-Tower:~/tpotce/installer/install$ ls
a.txt deploy.yml inventory.yml n.txt sudo.yml tpot.service tpot.yml
oppenheimer@abp-Tower:~/tpotce/installer/install$ cd ..
oppenheimer@abp-Tower:~/tpotce/installer$ cd ..
oppenheimer@abp-Tower:~/tpotce$ ls
CHANGELOG.md SECURITY.md docker genuser.sh uninstall.sh
CITATION.cff compose docker-compose.yml genuserwin.ps1 update.sh
LICENSE deploy.sh dps.ps1 install.sh version
README.md doc env.example installer
oppenheimer@abp-Tower:~/tpotce$ sudo ./install.sh
This script should not be run as root. Please run it as a regular user.
```

You will go in the installation directory tpotce and find an install.sh file, you have to run it through The normal user and not root.

3- Choosing -h as hive is the complete Tpot install medium

```
This script should not be run as root. Please run it as a regular user.

oppenheimer@abp-Tower:~/tpotce$ sudo ./install.sh --type=user
Usage: ./install.sh [-s] -t <type> [-u <webuser>] [-p <password>]

Options:
  -s                Suppress installation confirmation prompt (sets myQST=y)
  -t <type>         Type of installation (required if -s is used):
                    h - hive          (requires -u and -p)
                    s - sensor       (no user/pass required)
                    l - llm          (requires -u and -p)
                    i - mini         (requires -u and -p)
                    m - mobile       (no user/pass required)
                    t - tarpit       (requires -u and -p)
  -u <webuser>      Web interface username (required for h/l/i/t)
  -p <password>     Web interface password (required for h/l/i/t)
  -h                Show this help message
```

11 [Show error help message](#)

|   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |

```
### Install? (y/n) y
```

```
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

`sudo systemctl status tpot`

```

✓ miniprint Pulled 32.8s
✓ fatt Pulled 61.2s
✓ elasticpot Pulled 36.7s
✓ ciscoasa Pulled 32.8s
✓ sentrypeer Pulled 32.8s
✓ tanner_phpox Pulled 41.8s
✓ logstash Pulled 95.0s
✓ dicompot Pulled 27.6s
✓ snare Pulled 52.2s
✓ kibana Pulled 114.3s

### Please review for possible honeypot port conflicts.
### While SSH is taken care of, other services such as
### SMTP, HTTP, etc. might prevent T-Pot from starting.

Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User        Ino
de      PID/Program name
tcp      0      0 0.0.0.0:64295          0.0.0.0:*              LISTEN      0           436
89      16399/sshd: /usr/sb
tcp      0      0 0.0.0.0:22            0.0.0.0:*              LISTEN      0           642
1      11495/sshd: /usr/sb
tcp6     0      0 :::64295               :::*                   LISTEN      0           436
91      16399/sshd: /usr/sb
tcp6     0      0 :::22                  :::*                   LISTEN      0           729
2      11495/sshd: /usr/sb
udp      0      0 172.16.0.4:68         0.0.0.0:*              998        156
15      3053/systemd-networ
udp      0      0 127.0.0.1:323         0.0.0.0:*              0          778
9      1098/chronyd
udp6     0      0 :::1:323              :::*                   0          779
0      1098/chronyd

### Done. Please reboot and re-connect via SSH on tcp/64295.

```

## ***Deploying Financial Decoy:***

To enhance the realism of the Honeypot, I deployed a financial decoy right next to it , which is a WEB/API decoy. This simulates an online banking login portal and money transfer API endpoint ,Allowing attackers to interact with a fake financial Application. All activity I slogged for analysis.

1- Create the decoy directory inside honeypot VM :

*mkdir ~/financial-decoy*

*cd ~/financial-decoy*

2- Deploying Financial WEB decoy:

*index.html* Frontend

*financial app.py* Backend

This lightweight flask-based web app was made to attach the login page and provide a fake api endpoint which logs.

- API interactions
- Injection attempts
- Suspicious parameters
- Login attempts

3- Dependencies by using A virtual Environment:

This is made to avoid affecting T-pot services, And is a python environment

*sudo apt install python3-venv -y*

*cd ~/financial-decoy*

*python3 -m venv venv*

*source venv/bin/activate*

*pip install flask*

4- Running the financial decoy:

To keep it running over even when after terminal is closed systemd was created

sudo nano /etc/systemd/system/financial-decoy.service

The service configuration is used is :

[Unit]

Description=Financial Decoy Honeypot

After=network.target

[Service]

User=hpmojiz

WorkingDirectory=/home/hpmojiz/financial-decoy

ExecStart=/home/hpmojiz/financial-decoy/venv/bin/python3

/home/hpmojiz/financial-decoy/financial\_app.py

Restart=always

[Install]

WantedBy=multi-user.target

Activating the service:

This was made possible by enabling the daemon

sudo systemctl daemon-reload

sudo systemctl enable financial-decoy

sudo systemctl start financial-decoy

## **Network Rules for Both Vms:**

You can set these after going in your Specific VM and than Choosing networking in side options.  
Go to network settings.

## ***Tpot- Honeypot / Financial decoy:***

Inbound port rules:

| Prio... ↑ | Name                                                                                               |                                                                                   | Port        | Protocol |
|-----------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------|----------|
| ▼         | Inbound port rules (7)                                                                             |                                                                                   |             |          |
| 100       | SSH_HP                                                                                             |  | 64295       | TCP      |
| 110       | T_POT_web_ui                                                                                       |  | 64297       | TCP      |
| 120       |  all_danger_ports |  | 0-64293     | Any      |
| 130       | Allow_danger_port_2                                                                                |  | 64298-65... | Any      |
| 65000     | AllowVnetInBound ⓘ                                                                                 |  | Any         | Any      |
| 65001     | AllowAzureLoadBalancerInB... ⓘ                                                                     |  | Any         | Any      |
| 65500     | DenyAllInBound ⓘ                                                                                   |  | Any         | Any      |
| >         | Outbound port rules (3)                                                                            |                                                                                   |             |          |

- 1- SSH-HP will help us out to ssh in it through our management vm.
- 2- T\_POT\_web\_ui is the public facing port , Honeypot is deployed here.
- 3- We open all other ports as this is our decoy and we want attackers here.

## ***Management VM:***

Inbound port rules:

1- SSH , Only this as we SSH in management VM from our local pc To further SSH in our private VNET.

| Prio... ↑               | Name                           |                                                                                   | Port | Protocol |
|-------------------------|--------------------------------|-----------------------------------------------------------------------------------|------|----------|
| Inbound port rules (4)  |                                |                                                                                   |      |          |
| 100                     | SSH                            |  | 22   | TCP      |
| 65000                   | AllowVnetInBound ⓘ             |  | Any  | Any      |
| 65001                   | AllowAzureLoadBalancerInB... ⓘ |  | Any  | Any      |
| 65500                   | DenyAllInBound ⓘ               |  | Any  | Any      |
| Outbound port rules (3) |                                |                                                                                   |      |          |

2- Over here we set the source as MY IP Address , which for security reasons and as our laptops jump from Internet to Internet , We would have to change everytime, In an organizational scenario we don't have to worry about that.

Source ⓘ

IP Addresses

Source IP addresses/CIDR ranges \* ⓘ

95.183.227.33

Source port ranges \* ⓘ

\*

Destination ⓘ

Any

Service ⓘ

SSH

Destination port ranges ⓘ

22

Protocol

☐ Any

☒ TCP

☐ UDP

☐ ICMPv4

☐ ICMPv6

Action

☒ Allow

☐ Deny

Priority \* ⓘ

100

Save

Cancel

 Give feedback

Network Exposure Summary :



| VM      | Interface             | Address             | Exposed Ports                                  | Source Allowed    | Why                                     |
|---------|-----------------------|---------------------|------------------------------------------------|-------------------|-----------------------------------------|
| mgmt-vm | Public                | 4.248.187.160       | 22 (SSH)                                       | <b>My IP only</b> | Prevent public admin compromise         |
| mgmt-vm | Private               | 10.0.2.4            | N/A                                            | VNET internal     | Jump box routing                        |
| tpot-vm | Public                | 130.107.48.120      | 64297 (T-Pot UI),<br>plus deception ports      | Internet (decoy)  | Attract attackers and collect telemetry |
| tpot-vm | Private               | 10.0.1.4            | SSH from mgmt-vm<br>(custom port mapping used) | VNET only         | Prevent direct public admin             |
| tpot-vm | Public<br>(decoy app) | 130.107.48.120:8085 | 8085 HTTP                                      | Internet          | Financial decoy endpoint                |

## Accessibility Overview:

This section provides a clear walk-through of how the cloud honeypot system was deployed and how it can be accessed. You follow each step As given for Access and usage.

## **Management VM Access overview:**

Only the Management VM can be used to SSH into any host. Public management is intentionally restricted to prevent compromise of services.

So as we set my IP as the only IP which can access server you can see using Management VM's public IP I used my username and Password to access my MGMT VM through my local computer.

Adresses:

1- Public IP address: 4.248.187.160

2- Private IP address: 10.0.2.4

```
mojiz@Oppenheimer:~$ ssh mojiz@4.248.187.160
mojiz@4.248.187.160's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1012-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Dec  5 10:31:07 UTC 2025

System load:  0.0               Processes:            135
Usage of /:   9.7% of 28.02GB   Users logged in:     1
Memory usage: 6%               IPv4 address for eth0: 10.0.2.4
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

24 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Thu Dec  4 11:16:24 2025 from 95.183.227.33
mojiz@mgmt-vm:~$
```

### ***Honeypot VM Access overview:***

So this VM cant be configured or directly SSH into through its public IP, Its public IP is specified for Honeypot usage only.

We can only manage this Using our private VNET and SSH has to be done in the Management VM for this.

Adresses:

1- Public IP address: 130.107.48.120

2- Priavte IP address: 10.0.1.4

```
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

24 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Dec  5 10:54:16 2025 from 95.183.227.33
hpmojiz@mgmt-vm:~$ ssh -p 64295 hpmojiz@10.0.1.4
hpmojiz@10.0.1.4's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1014-azure x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Thu Dec  4 11:17:16 UTC 2025

System load:  0.37           Temperature:   49.9 C
Usage of /:   52.2% of 28.0GB Processes:      332
Memory usage: 93%           Users logged in: 0
Swap usage:   0%            IPv4 address for eth0: 10.0.1.4

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Dec  4 11:17:16 2025 from 10.0.2.4
hpmojiz@tpot-vm:~$
```

**Honeypot Ports and services:**

T-pot automatically runs the scripts and deploys multiple honeypots on the server each having a distinct port, These are important in collecting attacker traffic.

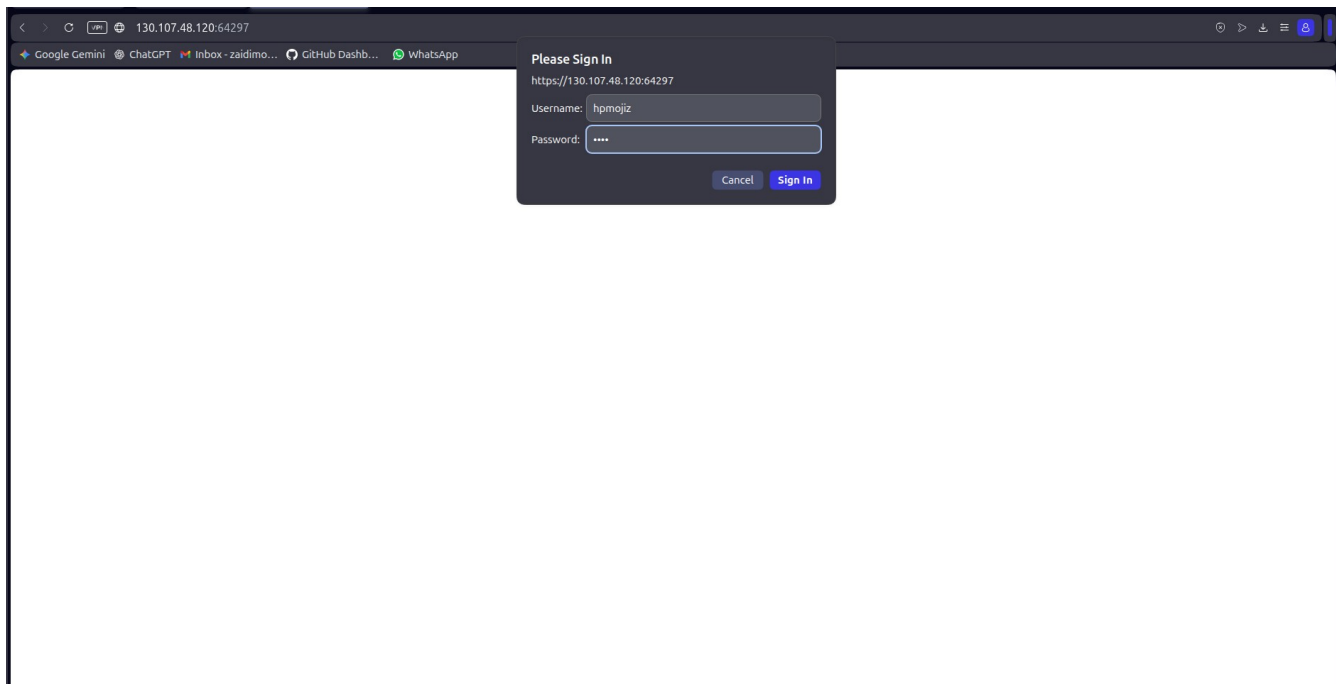
| Service Type      | Port  | Honeypot Component    | Purpose                          |
|-------------------|-------|-----------------------|----------------------------------|
| SSH               | 22    | Cowrie                | Brute force + credential attacks |
| Telnet            | 23    | Cowrie                | IoT botnet recruitment traps     |
| HTTP              | 80    | Various web honeypots | Scanning, injections             |
| HTTPS             | 443   | Web decoys            | Fake banking/API endpoints       |
| SMB               | 445   | SMB honeypot          | Lateral movement traps           |
| RDP-like          | 3389  | RDP decoy             | Windows-based attack simulation  |
| Elastic/Kibana UI | 64297 | Web UI                | Honeypot dashboard access        |

WE can access all these using the Public address of honeypot with the port specified as:

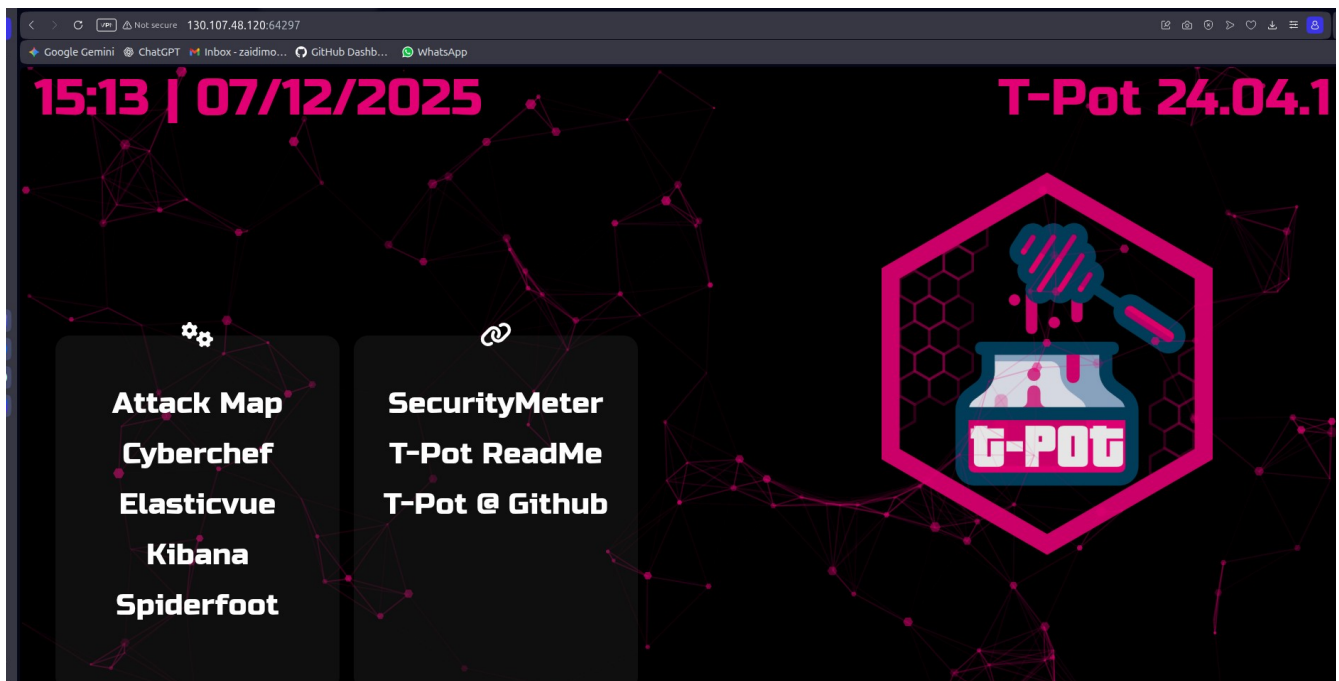
***https://<tpot-public-ip>:64297***

For this case:

***https://130.107.48.120:64297***



Here it asks for a username and password you created during T-Pot installation and then takes you to the T-Pot GUI.



The T-pot GUI is further divided into multiple Web interfaces, Kibana being the dashboard and Attack Map being the map of where and how attacks happened.

## **Financial Decoy:**

The financial decoy is now activated at port 8085 with a TCP

As the honeypot was allowed to take all inbound connections didn't have to change any network configurations for this.

Using the honeypot VM's public IP we can access it

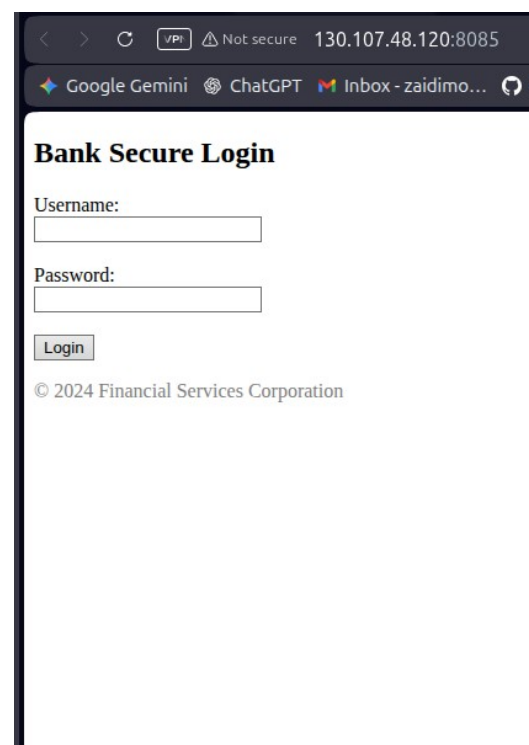
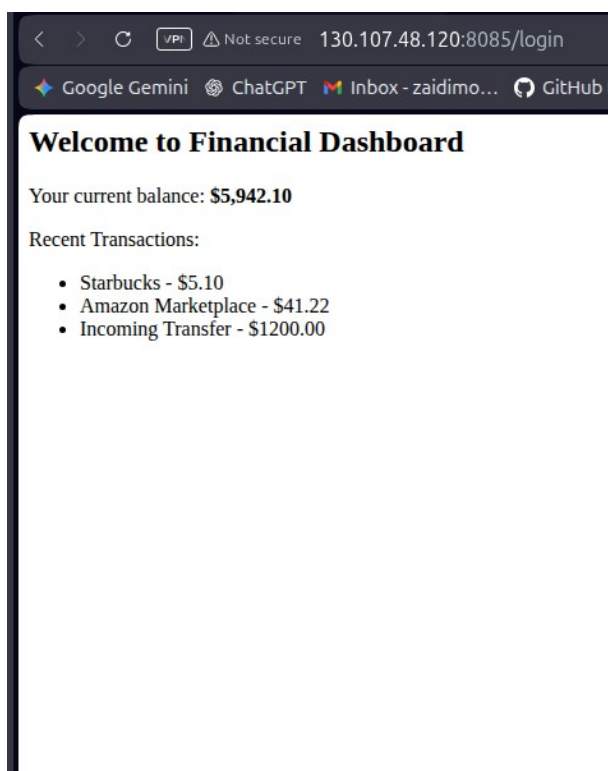
<http://130.107.48.120:8085>

Keep the HTTP header as attackers exploit This instead of HTTPS.

When attackers interact with this they will trigger:

- / Fake banking Login
- /login Fake dashboard showing details (Making them think they logged in)
- /api/transfer?amount=X^to=Y Fake transfer API endpoint

All these interactions are logged in Honeypot VM.



## **Operational maintenance and realism strategy:**

To keep decoys realistic overtime the environment should mimic plausible patterns :

- Realistic service banners and endpoints.
- Fake but consistent dummy data to avoid “Empty system” Indicators.
- Normal background traffic generation to avoid “Silent host” fingerprinting.

## Minimizing Human effort:

Human effort is reduced by :

- Centralizing telemetry in kibana for quick triage.
- High confidence signals , like any interaction with decoy is labeled suspicious.
- Automating enrichment (Extracting attacker IP and endpoint)
- Defining thresholds for response.

## Log ingestion Of financial decoy to Kibana:

The financial decoy writes HTTP access logs to

/var/log/financial-decoy.log.

We configure Firebeat to monitor this file and forward events to local elastic search instance used by T-POT. Events were indexed under the pattern financial-decoy-YYYY. MM. DD. A kibana data view Financial-Decoy-\* was created with @timestamp as time field to enable analysis in Discover.

You can follow these steps to configure the files for Kibana.

1. Confirm financial decoy is logging locally:

In tpot VM the access logs are written to /var/log/financial-decoy.log verify using

sudo tail -f /var/log/financial-decoy.log

After visiting http://<tpot-public-ip>:8085 and submitting a fake login you can see logs appear as

- GET/
- Post /Login
- Client Source IP in beginning of each line.

2. Identify elastic search port used by T-POT:

Find port using docker ps | grep elastic

Example: 0.0.0.0:64298 → 9200/tcp

so , The port is 64298 for ElasticSearch

Endpoint will be visited as

http://localhost:64298

3. Configure Filebeat to send decoy logs to ElasticSearch:

Filebeat was configured to read logs from /var/log/financial-decoy.log continuously and send events to Elasticsearch.

Open the config file : `sudo nano /etc/filebeat/filebeat.yml`

Input configuration in image

key points:

- *paths point to decoy log file*
- *Index forces a dedicated pattern*
- *fields under root adds app and log type*

*All these are done for easy filtering each interfaces logs*

```
yaml
filebeat.inputs:
  - type: log
    enabled: true
    paths:
      - /var/log/financial-decoy.log
    fields:
      app: financial-decoy
      log_type: financial-web-decoy
      fields_under_root: true

setup.template.enabled: false
setup.ilm.enabled: false

output.elasticsearch:
  hosts: ["localhost:64298"]
  index: "financial-decoy-%{+yyyy.MM.dd}"

logging.level: info
logging.to_files: true
```

#### 4. Validate & restart Filebeat:

Check syntax files: `sudo filebeat test config`

Restart: `sudo systemctl restart filebeat`

`sudo systemctl status filebeat`

Filebeat should show : active running

#### 5. Check if data arrived in ElasticSearch:

List indices to check Financial decoy exists:

`curl -s "http://localhost:64298/_cat/indices?v" | grep financial`

Verify sample documents:

`curl "http://localhost:64298/financial-decoy-*/_search?size=5&pretty"`

Now the decoy logs are being stored in ElasticSearch.

#### 6. Creating Kibana Dataview:

Open kibana: `https://<tpot-public-ip>:64297`

then :

- Stack Management → Data Views
- Create the Data View
- Index pattern: `financial-decoy-*`
- Timestamp Filed: `@timestamp`
- Save

Now kibana discover can query all decoy logs

#### 7. Create runtime field: attacker\_ip:

- Stack Managemnt → Data Views → Financial\_decoy → Addfield\*



- Use : Name: attacker\_ip, Type: IP
- add this in runtime script:

```
if (params._source.containsKey('message')) {
  def msg = params._source.message;
  if (msg != null) {
    def m = /(\d{1,3}(\?:\.\d{1,3}){3})/.matcher(msg);
    if (m.find()) {
      emit(m.group(1));
    }
  }
}
```

This extracts the first IPV4 address from the log line.

8. Create runtime

field: http\_method:

- Use : Name: http\_method, Type: keyword

```
if (params._source.containsKey('message')) {
  def msg = params._source.message;
  if (msg != null) {
    def m = /"([A-Z]+)\s+(\^[ ]+)\s+HTTP\/[0-9.]+"/.matcher(msg);
    if (m.find()) {
      emit(m.group(1));
    }
  }
}
```

9. Create runtime field: url\_path:

- Use : Name: url\_path, Type: keyword

```
if (params._source.containsKey('message')) {
  def msg = params._source.message;
  if (msg != null) {
    def m = /"([A-Z]+)\s+(\^[ ]+)\s+HTTP\/[0-9.]+"/.matcher(msg);
    if (m.find()) {
      emit(m.group(2));
    }
  }
}
```

10. Use runtime fields in Discover:

- Discover → select data view financial-decoy-\*
- Columns add:

attacker-ip , http\_method , url\_path

Now each event shows

which attacker ip accessed decoy, which endpoint was used. Which method was accessed

This structure will help in automating NSG rules , and check threshold based ip extraction as well as generating candidate data for deny rules to be put in azure.

| Setting    | Value                                             | Why                                   |
|------------|---------------------------------------------------|---------------------------------------|
| input path | /var/log/financial-decoy.log                      | reads decoy access logs               |
| output     | localhost:64298                                   | ships to T-Pot Elasticsearch          |
| index      | financial-decoy-%{+yyyy.MM.dd}                    | daily index management and separation |
| fields     | app=financial-decoy, log_type=financial-web-decoy | filtering and correlation             |

```
Financial Decoy (Flask :8085)
|
| writes access logs
v
/var/log/financial-decoy.log
|
| Filebeat input
v
Elasticsearch (localhost:64298)
|
| Kibana Data View: financial-decoy-*
v
Kibana Discover (https://130.107.48.120:64297)
|
| Runtime fields: attacker_ip, http_method, url_path
v
Attack visibility + future NSG automation input
```

